Lab 7 Procedure: Bode Plots for a 3D Printed Beam Attached to a DC Motor

Overview

The primary goal of this lab is to generate a bode plot for the input/output relationship of a 3D printed beam attached to a DC motor. The system will ultimately have two sensors: an encoder attached to the motor and an accelerometer attached to the tip of the beam. You will initially focus on control of the motor with the beam detached. Once you have that working, you will reattach the beam and the accelerometer.

AI Generation of Arduino Code

Dr. Krauss' Python-based wxbd_gui software has some quirks and bugs that can make it a little frustrating. This lab activity will be completed without the wxbd_gui software in order to decide if Dr. Krauss should continue maintaining and improving his software.

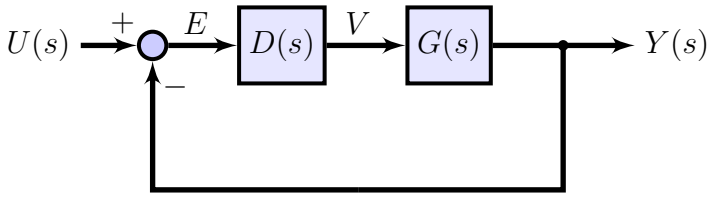
Disclaimer

If you have a strong aversion to using AI for generating Arduino code, you can discuss with Dr. Krauss starting from his template and writing your own Arduino code.

Feedback control of the DC motor without the beam

As discussed in last week's lab, generating a bode plot from experimental data requires using the fft algorithm. The fft requires that the experimental data start and end at the same point. Because a DC motor is not strictly open-loop stable, feedback control is required to perform Bode testing on this system.

So, your first task is to generate Arduino code that controls the DC motor based on encoder feedback:



Details and Context for AI Code Generation

Here are some details/context that the AI will need to generate code that actually does what you need:

- The AI should not be told about my template file. We need to see how much it can figure out on its own.
- The test needs to run for 5 seconds or so

- The encoder is connected to pins 2 and 11
  - 2 is channel A and 11 is channel B
- The motor is driven by an H-bridge that is attached to pins 6, 4, and 9
  - 9 is the PWM pin that controls the motor's speed
  - Pins 6 and 4 are the other input pins that control the motor's direction
- The most common issue with setting up a motor/encoder system is getting the signs right.
  - A positive step input must cause the encoder to increase so that the error is reduced.
  - If the code has pins 6 and 4 reversed, the motor will spin in the wrong direction and the error will increase.
- You need the Arduino to execute the control loop once every 2 milliseconds.
- The Arduino must print time along with the other input and output signals to the serial monitor every time step.
- You should use PD control with Kp=1 and Kd=0.1 as a starting point.
- You will need to include saturation before sending the PWM signal to the H-bridge

Note: Sign Errors

If you motor spins for the entire test, it is likely heading in the wrong direction as mentioned above. Tell the AI that this happened and that the sign on the encoder needs to switch.

Attaching the Beam and the Accelerometer

Once you are able to run a stable step input test, you are ready to re-attach the beam and connect the i^2c wires for the accelerometer.

New AI Info

The AI will need to update the Arduino code to read the accelerometer and print out its output to the serial monitor. Some details:

- we are using an MPU 6050 accelerometer/gyroscope
- we need to read the acceleration in the z direction
- I recommend setting the accelerometer's range to 8g's

Thread Note

Note: Please be careful not to strip the threads in the aluminum hubs. Some are SAE #4-40 and some are M3. These look very similar. Please try finger tightening one screw and then the other to verify.

Hub pic 1



Hub pic 2

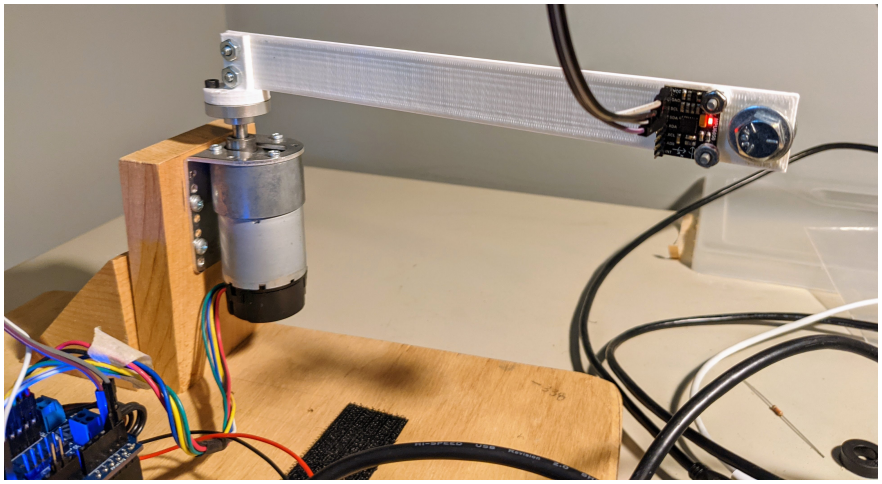


Beam attached to motor

The picture below shows a beam attached to a motor. Do not attach the accelerometer wires yet (if there are wires connected to your accelerometer, leave them dangling but do not attach

them to the Arduino yet).

Note that there are four washers with the 1/4-20 bolt at the end for weight. This is necessary to get the beam vibration modes at frequencies we can work with.



Initial tests *without* accelerometer wires

Run at least one initial swept-sine test without the accelerometer wires attached. If something goes haywire, we don't want the wires being ripped out of the Arduino.

Accelerometer Wiring

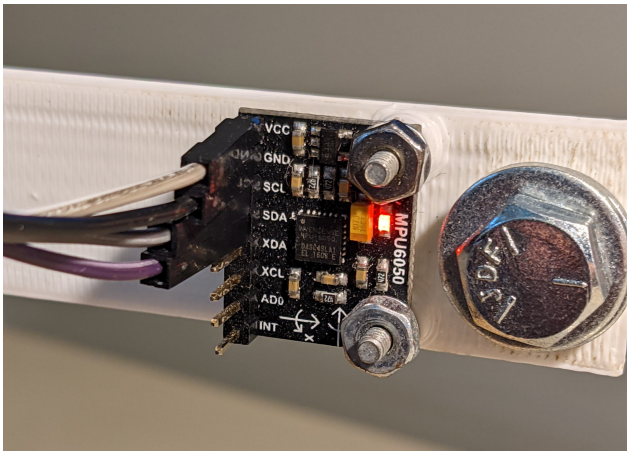
Once you have a good idea of what parameters to use for a test and have verified that the system is stable and nothing breaks, you are ready to attach the accelerometer wires.

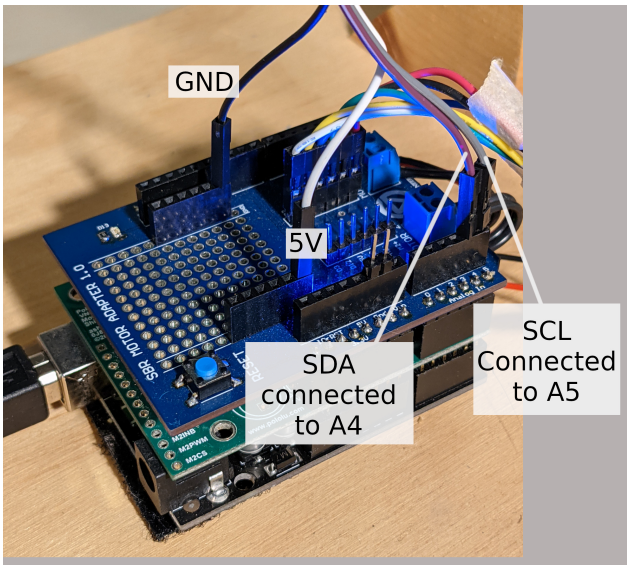
There are four wiring connections that need to be made between the MPU 6050 accelerometer and the Arduino:

Accel Pin	Arduino Pin
Vcc	5V
GND	GND
SCL	A5
SDA	A4

i^{2}c gotcha

After you connect the i^{2}c wires, be sure to restart the Arduino. If the Arduino does not detect any connected i^{2}c devices when it starts up, it gives up on trying to communicate over i^{2}c. If your accelerometer is giving all zeros in the serial monitor/data file, this is the most likely reason.





New Arduino Libraries

I will be curious to see how the AI handles the MPU 6050 and if it warns you about any necessary Arduino libraries. If your AI does not tell you to install any libraries, please try compiling your AI Arduino code and make note of what happens.

The template for performing this lab using Dr. Krauss' wxbd_gui uses two new libraries: `I2Cdev` and `MPU6050`. They can be installed from this github page:

https://github.com/jrowberg/i2cdevlib/tree/master/Arduino

Unfortunately, the author didn't really package the code for us to grab just one or two libraries. You download all of his code as a zip (from here: https://github.com/jrowberg/i2cdevlib) and then copy the two folders we need. Ultimately, you need folders called `I2Cdev` and `MPU6050` in your `Arduino/libraries` folder along with all their content.

- `Arduino`
  - `libraries`
    - `I2Cdev`

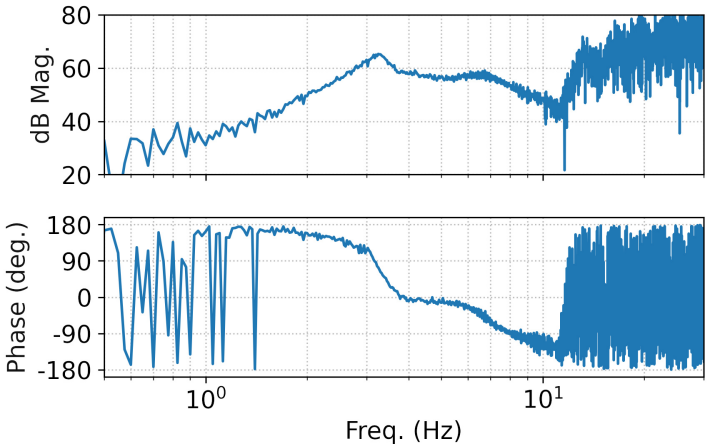
    - `I2Cdev.h`
    - `I2Cdev.cpp`
    - other stuff as well
* `MPU6050`
    - `MPU6050.h`
    - `MPU6050.cpp`
    - other stuff as well

Swept Sine Testing

Getting "good" swept sine data takes some trial and error along with tweaking the test parameters. Parameters to vary include the swept sine amplitude and slope, K_p and K_d, and `stop_t`. You know the data is "good" when it leads to a "good" Bode plot.

All Bode plots have noise at high frequencies. An accelerometer is subject to noise at low frequencies as well. So, we expect the Bode plot to have noise on the left and right sides.

The beam has two modes of vibration in the frequency range we are working with. This means we should see two peaks in Bode plot. Here is an example of a pretty good Bode plot



Example Mediocre Data

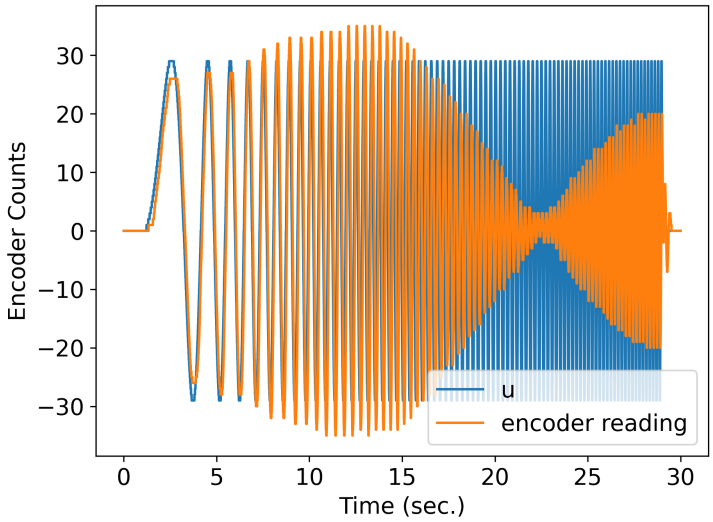
Below are a video and plots of swept sine data quickly gather by Dr. Krauss to help you know if you are in the ball park.

Swept Sine Video

Here is a video of a mediocre swept sine test to calibrate your expectations:

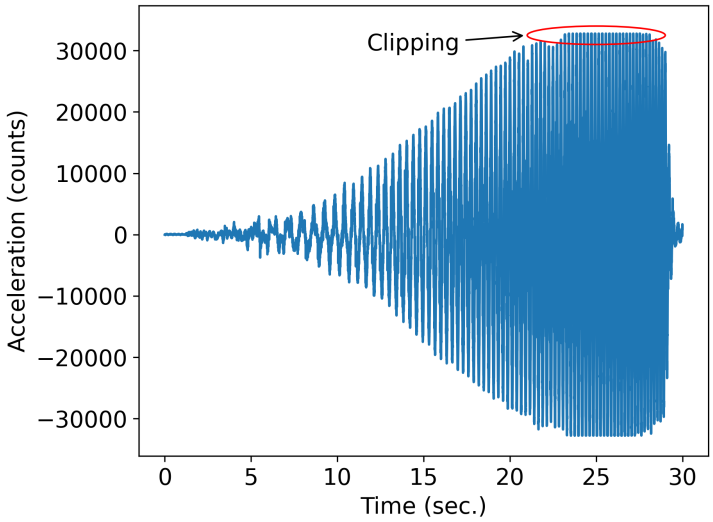
https://youtu.be/LIF4SnX-2LA

Closed-Loop Input and Encoder vs. Time



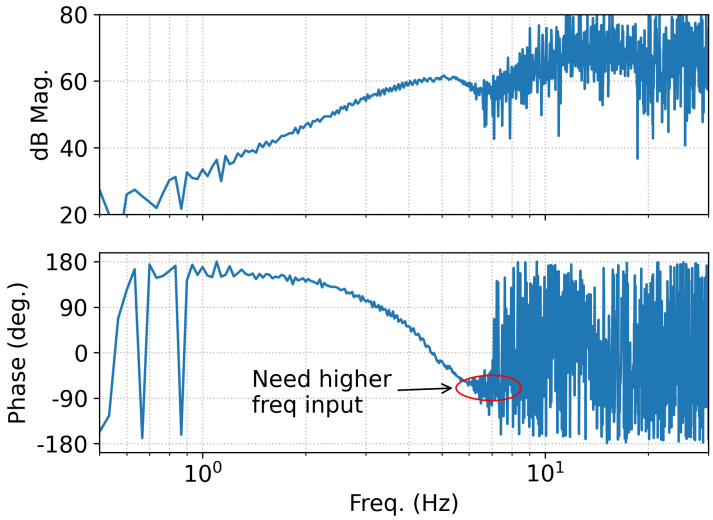
Clipping on the Acceleration Data

Dr. Krauss' acceleration data is mediocre because there is clipping. This means that the actual maximum acceleration was not measured correctly because it exceeded the measurement range of the sensor.



Less-Than-Mediocre Bode

The Bode plot based on Dr. Krauss' data is less than mediocre. The biggest issue is that the input does not reach a high enough frequency.



Deliverables: Bode Plots

Your memo must include two Bode plots:

$$G_1 = \frac{\text{encoder}}{U}$$

$$G_2 = \frac{\text{accelerometer}}{U}$$

where U is the closed-loop input in the block diagram shown above.

Comparison with 2DOF MSD Model

Once you have the Bode plot from the experimental data, I want you to start thinking about the transfer function that matches the Bode plot. As an individual homework assignment, you will find a TF that matches the Bode fairly closely. For this lab, investigate whether or not a 2DOF MSD system is a conceptual match to the Bode plot. You created 2DOF MSD TFs in LA 2. Reuse that code in this lab to create the TFs and then generate their Bode plots. Overlay the 2DOF Bode plots to see if they match the beam Bode. You will need to tweak the MSD parameters to see if you can get the Bodes fairly close.

Additionally, we know that differentiation in the time domain is the same as multiplying by s in the Laplace domain. So, you will almost certainly need to multiply the MSD TFs by s^2 to account for the fact that the sensor is an accelerometer.

Note: You do **not** need to use `optimize.fmin`; just get the Bode plots as close as you can through manual tweaking of coefficients.

Comprehension Questions

Q1: What did you observe during fixed sine tests near the two modes of vibration? What happens as the fixed sine frequency gets close to the vibration frequency? How are the responses at the two frequencies similar? How are the responses at the two frequencies different?

Q2: How did you arrive at the final form for your transfer function? What aspects of your Bode plot lead you to include certain terms in your TF?

- Q2 Note: discuss this for both TFs listed above G_1 and G_2
  - is there physical significance to any of the terms in your TFs?

Q3: What does a Bode plot represent?

Q4: When working with frequency response, what are magnitude ratio and phase shift?